

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ
по практической работе №3
по дисциплине «Анализ, моделирование и оптимизация систем»
Тема: Моделирование центра массового обслуживания с ограниченной
очередью.

Студент гр. 1306

Шаврин А.П.

Преподаватель

Мандрикова Б.С.

Санкт-Петербург

2025

Цель работы.

Целью работы является изучение модели обслуживания заявок с ограниченной очередью.

Для достижения поставленной цели требуется решить следующие задачи:

- 1) изучить модель обслуживания заявок со схожими законами распределения потока заявок и обслуживания;
- 2) изучить модель обслуживания заявок с различными законами распределения потока заявок и обслуживания;
- 3) запрограммировать модель;
- 4) сравнить практические результаты с теоретическими.

Постановка задачи.

Вариант 9: $\rho_1 = 0,60$; $0,85$, $N = 1500$; 47500

Необходимо смоделировать систему обслуживания заявок с ограниченной очередью с пуассоновским потоком заявок (время отправки сообщения — случайная величина, распределенная по экспоненциальному закону) и тремя различными потоками обслуживания (время обслуживания — случайная величина, распределенная по равномерному, показательному или треугольному закону).

Провести эксперимент и выяснить практические характеристики модели.

Провести теоретический расчет этих параметров.

Оценить результаты.

Выполнение работы.

Пусть интенсивность потока заявок $\lambda = \frac{1}{50} = 0,02$

Для равномерного закона с $\rho = 0,60$ и $m=2$

Интенсивность потока обслуживания $\mu = \frac{\lambda}{\rho} = \frac{0,02}{0,60} = 0,03(3)$

Среднее время обслуживания (FT) $\overline{t_{об}} = \frac{1}{\mu} = \frac{0,60}{0,02} = 30$

Коэффициент вариации времени обслуживания $\vartheta = \frac{\sigma_{t_{06}}}{\overline{t_{06}}} = \frac{\frac{x}{2\sqrt{3}}}{\frac{x}{2}} = \frac{1}{\sqrt{3}}$

Вероятность отказа $p_{\text{отк}} = \frac{1-\rho}{1-\rho^{m+2}} \rho^{m+1} = 0.099$

Среднее число заявок в очереди (QA) $\bar{r} = \frac{\rho^2(1-\rho^m(m-m\rho+1))(1+\vartheta^2)}{2(1-\rho^{m+2})(1-\rho)} = 0.24$

Среднее время ожидания в очереди (QT) $\overline{t_{\text{ож}}} = \frac{\rho^2(1-\rho^m(m-m\rho+1))(1+\vartheta^2)}{2\lambda(1-\rho^{m+2})(1-\rho)} = 13.47$

Таблица 1 – равномерный закон с $\rho = 0,60$

N	QM	QA	QZ	QT	QX	FR	FT	R	Ротк
1500	2	0.19	1166	8.36	48.5	0.563	29.805	91	0.061
47 500	2	0.23	36004	12.9	77.7	0.559	29.94	4320	0.091
Теория		0.24		13.47			30		0.099

В результате моделирования было получено, что FT, QT, QA близки к теоретическим значениям.

Для равномерного закона с $\rho = 0,85$ и $m = 5$

Интенсивность потока обслуживания $\mu = \frac{\lambda}{\rho} = \frac{0,02}{0,85} = 0,0235$

Среднее время обслуживания (FT) $\overline{t_{06}} = \frac{1}{\mu} = \frac{0,85}{0,02} = 42.5$

Коэффициент вариации времени обслуживания $\vartheta = \frac{\sigma_{t_{06}}}{\overline{t_{06}}} = \frac{\frac{x}{2\sqrt{3}}}{\frac{x}{2}} = \frac{1}{\sqrt{3}}$

Вероятность отказа $p_{\text{отк}} = \frac{1-\rho}{1-\rho^{m+2}} \rho^{m+1} = \frac{1-0.85}{1-0.85^7} 0.85^6 = 0.083$

Среднее число заявок в очереди (QA) $\bar{r} = \frac{\rho^2(1-\rho^m(m-m\rho+1))(1+\vartheta^2)}{2(1-\rho^{m+2})(1-\rho)} = 1.06$

Среднее время ожидания в очереди (QT) $\overline{t_{\text{ож}}} = \frac{\rho^2(1-\rho^m(m-m\rho+1))(1+\vartheta^2)}{2\lambda(1-\rho^{m+2})(1-\rho)} = 57.6$

Таблица 2 – равномерный закон с $\rho = 0,85$

N	QM	QA	QZ	QT	QX	FR	FT	R	Ротк
1500	5	0.85	626	48.44	87.44	0.765	42.48	102	0.068
47 500	5	0.99	19006	57.072	101	0.793	42.59	3819	0.08
Теория		1.05		57.6			42.5		0.083

В результате моделирования было получено, что FT, QT, QA близки к теоретическим значениям.

Для экспоненциального закона с $\rho = 0,60$ и $m = 1$

$$\text{Интенсивность потока обслуживания } \mu = \frac{\lambda}{\rho} = \frac{0,02}{0,60} = 0,03(3)$$

$$\text{Среднее время обслуживания (FT) } \overline{t_{об}} = \frac{1}{\mu} = \frac{0,60}{0,02} = 30$$

$$\text{Коэффициент вариации времени обслуживания } \vartheta = \frac{\sigma_{t_{об}}}{\overline{t_{об}}} = \frac{x}{x} = 1$$

$$\text{Вероятность отказа } p_{отк} = \frac{1-\rho}{1-\rho^{m+2}} \rho^{m+1} = 0.1836$$

$$\text{Среднее число заявок в очереди (QA) } \bar{r} = \frac{\rho^2(1-\rho^m(m-m\rho+1))(1+\vartheta^2)}{2(1-\rho^{m+2})(1-\rho)} = 0.1836$$

$$\text{Среднее время ожидания в очереди (QT) } \overline{t_{ож}} = \frac{\rho^2(1-\rho^m(m-m\rho+1))(1+\vartheta^2)}{2\lambda(1-\rho^{m+2})(1-\rho)} = 11.25$$

Таблица 3 – экспоненциальный закон с $\rho = 0,60$

N	QM	QA	QZ	QT	QX	FR	FT	R	Ротк
1500	1	0.17	1114	10.23	81.1	0.53	30.3	225	0.15
47 500	1	0.182	34929	11.15	105.7	0.54	30.06	8449	0.178
Теория		0.1836		11.25			30		0.1836

В результате моделирования было получено, что FT, QT, QA близки к теоретическим значениям.

Для экспоненциального закона с $\rho = 0,85$ и $m = 5$

$$\text{Интенсивность потока обслуживания } \mu = \frac{\lambda}{\rho} = \frac{0,02}{0,85} = 0,0235$$

$$\text{Среднее время обслуживания (FT) } \overline{t_{об}} = \frac{1}{\mu} = \frac{0,85}{0,02} = 42.5$$

$$\text{Коэффициент вариации времени обслуживания } \vartheta = \frac{\sigma_{t_{об}}}{\overline{t_{об}}} = \frac{x}{x} = 1$$

$$\text{Вероятность отказа } p_{отк} = \frac{1-\rho}{1-\rho^{m+2}} \rho^{m+1} = 0.083$$

$$\text{Среднее число заявок в очереди (QA) } \bar{r} = \frac{\rho^2(1-\rho^m(m-m\rho+1))(1+\vartheta^2)}{2(1-\rho^{m+2})(1-\rho)} = 1.585$$

$$\text{Среднее время ожидания в очереди (QT) } \overline{t_{ож}} = \frac{\rho^2(1-\rho^m(m-m\rho+1))(1+\vartheta^2)}{2\lambda(1-\rho^{m+2})(1-\rho)} = 86.43$$

Таблица 4 – экспоненциальный закон с $\rho = 0,85$

N	QM	QA	QZ	QT	QX	FR	FT	R	Ротк
1500	5	1.62	581	91.5	157.6	0.772	41.48	114	0.076
47 500	5	1.54	18524	90.18	156.5	0.788	42.2	3752	0.079
Теория		1.585		86.43			42.5		0.083

В результате моделирования было получено, что FT, QT, QA близки к теоретическим значениям.

Для треугольного закона с $\rho = 0,60$ и $m = 2$

$$\text{Интенсивность потока обслуживания } \mu = \frac{\lambda}{\rho} = \frac{0,02}{0,60} = 0,03(3)$$

$$\text{Среднее время обслуживания (FT) } \overline{t_{06}} = \frac{1}{\mu} = \frac{0,60}{0,02} = 30$$

$$\text{Коэффициент вариации времени обслуживания } \vartheta = \frac{\sigma_{t_{06}}}{\overline{t_{06}}} = \frac{\sqrt{\frac{x^2}{6}}}{x} = \frac{1}{\sqrt{6}}$$

$$\text{Вероятность отказа } p_{\text{отк}} = \frac{1-\rho}{1-\rho^{m+2}} \rho^{m+1} = 0.099$$

$$\text{Среднее число заявок в очереди (QA) } \bar{r} = \frac{\rho^2(1-\rho^m(m-m\rho+1))(1+\vartheta^2)}{2(1-\rho^{m+2})(1-\rho)} = 0.21$$

$$\text{Среднее время ожидания в очереди (QT) } \overline{t_{\text{ож}}} = \frac{\rho^2(1-\rho^m(m-m\rho+1))(1+\vartheta^2)}{2\lambda(1-\rho^{m+2})(1-\rho)} = 11.7$$

Таблица 5 – треугольный закон с $\rho = 0,60$

N	QM	QA	QZ	QT	QX	FR	FT	R	Ротк
1500	2	0.198	1155	9.22	55.5	0.562	29.915	115	0.077
47 500	2	0.235	36453	13.23	87.1	0.568	30.03	4520	0.095
Теория		0.24		13.47			30		0.099

В результате моделирования было получено, что FT, QT, QA близки к теоретическим значениям.

Для треугольного закона с $\rho = 0,85$ и $m = 5$

$$\text{Интенсивность потока обслуживания } \mu = \frac{\lambda}{\rho} = \frac{0,02}{0,85} = 0,0235$$

$$\text{Среднее время обслуживания (FT) } \overline{t_{06}} = \frac{1}{\mu} = \frac{0,85}{0,02} = 42.5$$

$$\text{Коэффициент вариации времени обслуживания } \vartheta = \frac{\sigma_{t_{06}}}{\overline{t_{06}}} = \frac{\sqrt{\frac{x^2}{6}}}{x} = \frac{1}{\sqrt{6}}$$

$$\text{Вероятность отказа } p_{\text{отк}} = \frac{1-\rho}{1-\rho^{m+2}} \rho^{m+1} = 0.083$$

$$\text{Среднее число заявок в очереди (QA)} \bar{r} = \frac{\rho^2(1-\rho^m(m-m\rho+1))(1+\vartheta^2)}{2(1-\rho^{m+2})(1-\rho)} = 0.92$$

$$\text{Среднее время ожидания в очереди (QT)} \bar{t}_{\text{ож}} = \frac{\rho^2(1-\rho^m(m-m\rho+1))(1+\vartheta^2)}{2\lambda(1-\rho^{m+2})(1-\rho)} = 50.41$$

Таблица 6 – треугольный закон с $\rho = 0,85$

N	QM	QA	QZ	QT	QX	FR	FT	R	Ротк
1500	5	0.88	613	48.38	86.9	0.821	43.23	117	0.077
47 500	5	0.91	19657	49.99	91.1	0.812	42.49	3899	0.082
Теория		0.92		50.41			42.5		0.083

В результате моделирования было получено, что FT, QT, QA близки к теоретическим значениям.

Для экспоненциального закона с $\rho = 0,60$ и $N = 100\,000$ проведем 10 экспериментов и рассчитаем среднее время ожидания заявки в очереди и СКО.

Таблица 7. – экспоненциальный закон с $\rho = 0,60$ и $N = 100\,000$

№	1	2	3	4	5	6	7	8	9	10
QT	11.047	11.152	11.218	11.324	11.24	11.356	11.189	11.412	11.203	11.259
$P_{\text{отк}}$	0.1861	0.1829	0.1868	0.1912	0.1824	0.1824	0.1915	0.1874	0.1813	0.1863

Теоретическое время ожидания в очереди: $QT = 11.25$

Практическое среднее время ожидания в очереди: $QT_{\text{ср}} = 11.24$

$$\text{СКО: } \sigma(x) = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - x_{\text{ср}})^2} = 0.133$$

Доверительный интервал при 95%, критическое значение 2.262

$$QT_{\text{ср}} \pm 2.262 \frac{\sigma}{\sqrt{n}} = 11.24 \pm 0.042$$

Теоретическое значение соответствует полученному интервалу.

Теоретическая вероятность отказа: $P_{\text{отк}} = 0.1836$

Практическая вероятность отказа: $P_{\text{ср}} = 0.18583$

$$\text{СКО: } \sigma(x) = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - x_{\text{ср}})^2} = 0.00361$$

Доверительный интервал при 95%, критическое значение 2.262

$$QT_{\text{cp}} \pm 2.262 \frac{\sigma}{\sqrt{n}} = 0.1836 \pm 0.0026$$

Теоретическое значение соответствует полученному интервалу.

Выводы.

В ходе работы была успешно смоделирована система массового обслуживания с ограниченной очередью и исследовано влияние различных законов распределения времени обслуживания на характеристики системы.

Точность моделирования подтверждена высоким соответствием практических результатов теоретическим расчетам.

Проведенная серия из 10 экспериментов подтвердила статистическую значимость результатов: теоретическое значение времени ожидания (11.25) попадает в 95% доверительный интервал (11.24 ± 0.042), а теоретическое значение вероятности отказа (0.1836) попадает в 95% доверительный интервал (0.1836 ± 0.0026), что свидетельствует о корректности модели и отсутствии систематической погрешности.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД

```
from typing import List

import simpy
import numpy as np
import pandas as pd
from enum import Enum, auto
from dataclasses import dataclass

np.random.seed(42)

pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)
pd.set_option('display.max_colwidth', None)

class Distribution(Enum):
    UNIFORM = auto()
    EXPONENTIAL = auto()
    TRIANGULAR = auto()

@dataclass
class ExperimentStats:
    # Характеристики очереди
    QM: int # Максимальная длина очереди (Queue Max)
    QA: float # Средняя длина очереди (Queue Average)
    QZ: int # Число заявок, поступивших на обслуживание без
ожидания (Queue Zero)
    QT: float # Среднее время пребывания заявки в очереди
(включая нулевые ожидания)
    QX: float # Среднее время пребывания заявки в очереди
(без нулевых ожиданий)

    # Характеристики устройства (сервера)
```



```

        FR: float    # Коэффициент загрузки сервера (Facility Rate
/ Utilization)

        FT: float    # Среднее время обслуживания заявки (Facility
Time / Average Service Time)


rejects: float    # число отказов
reject_prob: float    # вероятность отказа


lambda_effective: float    # Эффективность с учетом отказов


# Дополнительно для анализа вариативности
std_wait_time: float = 0.0    # СКО времени ожидания
std_service_time: float = 0.0    # СКО времени обслуживания


# Параметры эксперимента
N: int = 0    # Общее число заявок, прошедших через систему
rho: float = 0.0    # Приведенная интенсивность потока ( $\lambda/\mu$ )
service_dist: Distribution = Distribution.UNIFORM


@dataclass
class UseCase:
    rho: float
    N: int
    service_dist: Distribution
    max_queue_len: int


@dataclass
class UseCaseGroup:
    name: str
    description: str
    use_cases: List[UseCase]


class TimeGenerator:
    def __init__(self, dist_type: Distribution, avg_time:
float):

```

```

        self.__dist_type: Distribution = dist_type
        self.__avg_time: float = avg_time

    def generate(self, count: int = 1) -> float:
        match(self.__dist_type):
            case Distribution.UNIFORM:
                return np.random.uniform(low=0,
high=self.__avg_time * 2, size=count)
            case Distribution.EXPONENTIAL:
                return np.random.exponential(scale=self.__avg_time, size=count)
            case Distribution.TRIANGULAR:
                return np.random.triangular(
                    left=0,
                    mode=self.__avg_time,
                    right=self.__avg_time * 2,
                    size=count
                )
            case _:
                raise ValueError("Unknown distribution type")

class ServiceSystem:
    def __init__(self, env, service_gen: TimeGenerator,
max_queue_len: int):
        self.env = env
        self.server = simpy.Resource(env, capacity=1)
        self.service_gen = service_gen
        self.max_queue_length = max_queue_len

        self.wait_times = []
        self.queue_lengths = []
        self.no_wait = 0
        self.rejects = 0
        self.service_times = []

```

```

def handle_request(self):
    """Процесс обслуживания заявки"""
    arrival_time = self.env.now
    queue_length = len(self.server.queue)
    if queue_length > self.max_queue_length:
        self.rejects += 1
        return
    without_wait = queue_length == 0
    self.queue_lengths.append(queue_length)

    with self.server.request() as req:
        yield req
        if not without_wait:
            self.wait_times.append((self.env.now
arrival_time)[0])
        else:
            self.no_wait += 1
            self.wait_times.append(0)
            service_time = self.service_gen.generate()
            self.service_times.append(service_time)
            yield self.env.timeout(service_time)

class RequestGenerator:
    def __init__(
        self,
        env: simpy.Environment,
        system: ServiceSystem,
        N: int,
        interarrival_gen: TimeGenerator,
    ):
        self.env = env
        self.system = system
        self.N = N
        self.interarrival_gen = interarrival_gen

```

```

def run(self):
    for i in range(self.N):
        interarrival = self.interarrival_gen.generate()
        yield self.env.timeout(interarrival)
        self.env.process(self.system.handle_request())

def run_experiment(N: int, mu: float, lamb: float,
service_dist: Distribution, max_queue_len: int):
    avg_service_time: float = 1 / mu
    avg_interval_time: float = 1 / lamb
    env = simpy.Environment()

    interarrival_gen = TimeGenerator(
        dist_type=Distribution.EXPONENTIAL,
        avg_time=avg_interval_time
    )
    service_gen = TimeGenerator(
        dist_type=service_dist,
        avg_time=avg_service_time
    )

    system = ServiceSystem(
        env=env,
        service_gen=service_gen,
        max_queue_len=max_queue_len
    )

    request_gen = RequestGenerator(
        env=env,
        system=system,
        N=N,
        interarrival_gen=interarrival_gen,
    )
    env.process(request_gen.run())
    env.run()

```

```

total_time: float = env.now

    QX      =      list(filter(lambda      time:      time      >      0,
system.wait_times))

return ExperimentStats(
    QM=max(system.queue_lengths),
    QA=np.mean(system.queue_lengths),
    QZ=system.no_wait,
    QT=np.mean(system.wait_times),
    QX=np.mean(QX) if len(QX) else 0.0,
    FR=sum(system.service_times) / total_time,
    FT=np.mean(system.service_times),
    rejects=system.rejects,
    reject_prob=system.rejects / N,
    lambda_effective=lamb * (1 - system.rejects / N),
)

def run_experiments(N: int, rho: float, lam: float,
service_dist: Distribution, max_queue_len: int, repeats: int) ->
ExperimentStats:
    mu: float = lam / rho

    all_stats: list[ExperimentStats] = []
    for _ in range(repeats):
        experiment_statistic = run_experiment(
            N=N,
            mu=mu,
            lamb=lam,
            service_dist=service_dist,
            max_queue_len=max_queue_len,
        )
        all_stats.append(experiment_statistic)

    return ExperimentStats(

```

```

        N=N,
        rho=rho,
        service_dist=service_dist,
        QM=int(np.mean(list(map(lambda stat: stat.QM,
all_stats))))),
        QA=float(np.mean(list(map(lambda stat: stat.QA,
all_stats))))),
        QZ=int(np.mean(list(map(lambda stat: stat.QZ,
all_stats))))),
        QT=float(np.mean(list(map(lambda stat: stat.QT,
all_stats))))),
        QX=float(np.mean(list(map(lambda stat: stat.QX,
all_stats))))),
        FR=float(np.mean(list(map(lambda stat: stat.FR,
all_stats))))),
        FT=float(np.mean(list(map(lambda stat: stat.FT,
all_stats))))),
        rejects=float(np.mean(list(map(lambda stat:
stat.rejects, all_stats))))),
        reject_prob=float(np.mean(list(map(lambda stat:
stat.reject_prob, all_stats))))),
        lambda_effective=float(np.mean(list(map(lambda stat:
stat.lambda_effective, all_stats))))),
        std_wait_time=float(np.std(list(map(lambda stat:
stat.QT, all_stats)), ddof=1)),
        std_service_time=float(np.std(list(map(lambda stat:
stat.FT, all_stats)), ddof=1))
    )

```

```

def main():
    lam: float = 1 / 50
    use_cases_groups = [
        UseCaseGroup(
            name="Равномерное распределение  $\rho=0.6$ ",

```

```

        description="Исследование для равномерного закона
обслуживания с низкой нагрузкой",
        use_cases=[
            UseCase(rho=0.6,                                N=1500,
service_dist=Distribution.UNIFORM, max_queue_len=1),
            UseCase(rho=0.6,                                N=47_500,
service_dist=Distribution.UNIFORM, max_queue_len=1),
        ]
    ),
    UseCaseGroup(
        name="Равномерное распределение  $\rho=0.85$ ",
        description="Исследование для равномерного закона
обслуживания с высокой нагрузкой",
        use_cases=[
            UseCase(rho=0.85,                                N=1500,
service_dist=Distribution.UNIFORM, max_queue_len=4),
            UseCase(rho=0.85,                                N=47_500,
service_dist=Distribution.UNIFORM, max_queue_len=4),
        ]
    ),
    UseCaseGroup(
        name="Экспоненциальное распределение  $\rho=0.6$ ",
        description="Исследование для экспоненциального
закона обслуживания с низкой нагрузкой",
        use_cases=[
            UseCase(rho=0.6,                                N=1500,
service_dist=Distribution.EXPONENTIAL, max_queue_len=1),
            UseCase(rho=0.6,                                N=47_500,
service_dist=Distribution.EXPONENTIAL, max_queue_len=1),
        ]
    ),
    UseCaseGroup(
        name="Экспоненциальное распределение  $\rho=0.85$ ",
        description="Исследование для экспоненциального
закона обслуживания с высокой нагрузкой",

```

```

        use_cases=[
            UseCase(rho=0.85, N=1500,
service_dist=Distribution.EXPONENTIAL, max_queue_len=5),
            UseCase(rho=0.85, N=47_500,
service_dist=Distribution.EXPONENTIAL, max_queue_len=5),
        ]
    ),
    UseCaseGroup(
        name="Треугольное распределение  $\rho=0.6$ ",
        description="Исследование для треугольного закона
обслуживания с низкой нагрузкой",
        use_cases=[
            UseCase(rho=0.6, N=1500,
service_dist=Distribution.TRIANGULAR, max_queue_len=1),
            UseCase(rho=0.6, N=47_500,
service_dist=Distribution.TRIANGULAR, max_queue_len=1),
        ]
    ),
    UseCaseGroup(
        name="Треугольное распределение  $\rho=0.85$ ",
        description="Исследование для треугольного закона
обслуживания с высокой нагрузкой",
        use_cases=[
            UseCase(rho=0.85, N=1500,
service_dist=Distribution.TRIANGULAR, max_queue_len=4),
            UseCase(rho=0.85, N=47_500,
service_dist=Distribution.TRIANGULAR, max_queue_len=4),
        ]
    ),
]

for group in use_cases_groups:
    print(f"\n{'=' * 60}")
    print(f"ГРУППА: {group.name}")
    group_results = []

```



```

for use_case in group.use_cases:
    stats = run_experiments(
        N=use_case.N,
        rho=use_case.rho,
        lam=lam,
        service_dist=use_case.service_dist,
        max_queue_len=use_case.max_queue_len,
        repeats=1
    )
    group_results.append(stats)

df_group_results = pd.DataFrame([s.__dict__ for s in
group_results])
df_group_results = df_group_results.set_index('N')
print(df_group_results)

if __name__ == "__main__":
    main()

```